

Preliminaries

For discrete case

$$\sum_{-\infty}^{\infty} p[x] = 1, P(X = x) = p[x]$$

$$E[X] = \sum_x xP(X = x) = \sum_x xp[x]$$

For continuous case

$$\int_{-\infty}^{\infty} p(x)dx = 1, P(a \leq X \leq b) = \int_a^b p(x)dx$$

$$E[X] = \int_{-\infty}^{\infty} xp(x)dx$$

Here, $P(\cdot)$ represents probability for a random variable taking a value or falling within a range of values, $p(\cdot)$ or $p[\cdot]$ represent probability density or mass functions for continuous or discrete observed variables (realizations), respectively. X is a random variable, and x is a specific realization of the random variable X . A realization of a random variable is the specific value that the variable takes in a particular trial or experiment. The horizontal axis of a PDF or a PMF represents realization x . Are $P(X), p(x), p[x]$ the same? Except for $P(X)$ denoting the population and $p(x), p[x]$ denoting the sample (or observed) values?

Optimization

Discrete optimization: Find the best discrete objects

$$\min_{p \in \mathcal{D}} \text{Cost}(p); \quad \mathcal{D} = \text{discrete set}$$

Continuous optimization: Find the best vector of real numbers

$$\min_{\vec{w} \in \mathbb{R}^d} \text{Cost}(\vec{w})$$

A demo problem: Computing edit distance. As inputs you have two strings s and t . You need to find the minimum number of edits it takes to change s into t . By 'edit' here, we mean any of the three operations: 1) insertion, 2) deletion, 3) substitution. How would you solve it? What is the optimal strategy? Is this an optimization problem? how? if yes, discrete? or continuous?

Consider another problem: Imagine you're managing a fleet of trucks, and fuel efficiency depends on the speed at which the truck travels. Driving too fast wastes fuel due to air resistance, while driving too slowly is inefficient due to the engine's lower performance. The goal is to identify the optimal speed that minimizes fuel usage which can be modeled as $a/v + bv^2$, with v being the speed. How would you solve it? Is this also an optimization problem? how? if yes, discrete? or continuous?

Machine Learning

Machine learning can be seen from three perspectives. 1) Modeling: what is the type of the machine learning model $h_w(x)$? 2) Learning: How are the model parameters determined given a set of training data? 3) Inference: Given a trained model, what is the predicted output for the test data? Here, the output y , called the target, can be discrete, continuous depending on the type of the problem. The input x , generally called the feature, can also vary based on the problem.

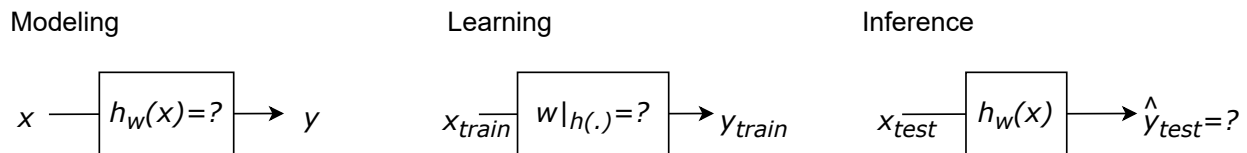


Figure 1: Three paradigms: Modeling, Learning, Inference.

Linear regression

If $\vec{x}$ represents a feature vector and y represents a continuous target variable, the linear regression model can be expressed mathematically as:

$$\hat{y} = h(\vec{x}) = \vec{w} \cdot \vec{x}; \quad \vec{w} = [w_1, w_2, \dots, w_d]^T$$

where, $w_1, \dots, w_d$ are the model parameters of the linear regression model.

Once, we have the model, the next step is to train the model, which means to compute the model parameters of the model, given a set of training data $\{(\vec{x}^{(1)}, y^{(1)}), (\vec{x}^{(2)}, y^{(2)}), \dots, (\vec{x}^{(m)}, y^{(m)})\}$. For that, it is a common practice to define an optimization problem and then solve that problem to find out the values of the model parameters. A commonly used loss (or cost) function is squared error loss as follows:

$$J(\vec{w}) = \frac{1}{2m} \sum_{i=1}^m \left(h(\vec{x}^{(i)}) - y^{(i)} \right)^2 = \frac{1}{2m} \sum_{i=1}^m \left(\vec{w} \cdot \vec{x}^{(i)} - y^{(i)} \right)^2$$

The optimization problem is then

$$\min_{\vec{w}} J(\vec{w}) = \min_{\vec{w}} \frac{1}{2m} \sum_{i=1}^m \left(\vec{w} \cdot \vec{x}^{(i)} - y^{(i)} \right)^2$$

and the optimum model parameter vector (i.e., the goal solution in training) is

$$\vec{w}^* = \arg \min_{\vec{w}} \frac{1}{2m} \sum_{i=1}^m \left(\vec{w} \cdot \vec{x}^{(i)} - y^{(i)} \right)^2$$

One way to solve the above optimization problem is to use gradient descent. Gradient descent starts with random initializations of $\vec{w}$, then iterative updates using the following update equation:

$$\vec{w} \leftarrow \vec{w} - \eta \nabla_{\vec{w}} J(\vec{w}) \quad \text{or} \quad \vec{w} \leftarrow \vec{w} - \eta \frac{\partial}{\partial \vec{w}} J(\vec{w}) \quad \text{or} \quad \vec{w} \leftarrow \vec{w} - \eta \frac{1}{m} \sum_{i=1}^m \left(h(\vec{x}^{(i)}) - y^{(i)} \right) \vec{x}^{(i)}$$

The parameter η represents the gradient descent step-size. The iteration stops when $\Delta J(\vec{w}) < \epsilon$.

Once we have the optimum model parameters $\vec{w}^*$, the next and the final step is inference, which is to use the model to predict the target of a test sample as follows:

$$\hat{y}_{test} = h(\vec{x}_{test}) = \vec{w}^* \cdot \vec{x}_{test}$$

Linear classification

If $\vec{x}$ represents a feature vector and y represents a discrete target variable, the linear classification model can be expressed mathematically as:

$$\hat{y} = h(\vec{x}) = \text{sign}(\vec{w} \cdot \vec{x}) \quad \text{or} \quad \hat{y} = h(\vec{x}) = \sigma(\vec{w} \cdot \vec{x}), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad (\text{sigmoid})$$

where, $w_1, \dots, w_d$ are the model parameters. Like $\text{sign}(\cdot)$ or $\sigma(\cdot)$ here, other nonlinear functions can also be used.

The next step naturally is to train the model or compute the model parameters of the model, given a set of training data $\{(\vec{x}^{(1)}, y^{(1)}), (\vec{x}^{(2)}, y^{(2)}), \dots, (\vec{x}^{(m)}, y^{(m)})\}$. For that, like the regression example above, it is a common practice to define an optimization problem first, and then solve that problem to find out the values of the model parameters. Once we have the optimum model parameters $\vec{w}^*$, the final step is inference, which is to use the model to predict the target of a test sample as follows:

$$\hat{y}_{test} = h(\vec{x}_{test}) = \text{sign}(\vec{w}^* \cdot \vec{x}_{test}) \quad \text{or} \quad \sigma(\vec{w}^* \cdot \vec{x}_{test})$$

Neural network: a cascade

In fact, we can expand the extension above a bit to build a concatenation of nonlinear computation units, which will become a simple form of what is widely known as neural networks. The modeling equation will then be following:

$$\hat{y} = h(\vec{x}) = \sum_j v_j \sigma(\vec{w}_j \cdot \vec{x})$$

We can further expand the extension above to build a series of computational units, add more components, architectures, which will merge toward a practically used neural network. Once we have the model, the idea of training, optimization, and inference are similar as before (may not be exactly the same).

Regularization

Add a penalty term with the cost function

$$J(\vec{w}) = J_{regular}(\vec{w}) + \frac{\lambda}{2} \|\vec{w}\|_{L_2}^2, \text{ where, } \|\vec{w}\|_{L_2}^2 = w_1^2 + \dots + w_d^2.$$

Unsupervised learning

Here, y_{train} also unknown. The goal to find structures in data. One example of unsupervised learning is clustering. Clustering is grouping of data samples based on their feature space. Let's take a look at a well known clustering algorithm: k -means clustering algorithm.

Let's assume we have m number of data points and we have to find out K clusters. We have d features, i.e., the feature vector $\vec{x}$ lives in a d -dimensional space, i.e., $\vec{x} \in \mathbb{R}^d$. Let $\vec{\mu}^{(k)} \in \mathbb{R}^d$ be the centroid of the k -th cluster. Our goal is to minimize the following loss function

$$J(\vec{\mu}, z) = \sum_{i=1}^m \|\vec{x}^{(i)} - \vec{\mu}^{(z_i)}\|_{L_2}^2$$

Here, we need to choose the centroid $\vec{\mu}$ and assignment z jointly. Here are the steps:

1. Randomly initialize the centroids $\vec{\mu}^{(k)}$, $k \in [1, K]$.
2. Assign each data point to the closest centroid:

$$z_i \leftarrow \arg \min_{k \in [1, K]} \|\vec{x}^{(i)} - \vec{\mu}^{(k)}\|_{L_2}^2, \quad i \in [1, m].$$

3. Recalculate centroids, set $\vec{\mu}^{(k)}$ equal to average of all assigned points:

$$\vec{\mu}^{(k)} \leftarrow \frac{1}{|\{i : z_i = k\}|} \sum_{i: z_i = k} \vec{x}^{(i)}, \quad k \in [1, K].$$

Here, $|\cdot|$ denotes the cardinality of a set.

Step 1 above will run only once. Next, we will run the updates in Steps 2 and 3 sequentially back and forth ($2 \rightarrow 3 \rightarrow 2 \rightarrow 3 \rightarrow \dots$) until convergence.

Search

So far, we've talked about machine learning models, which are reflex-based: one input leads to one output (single action). Now, let's discuss search models, which are state-based: a sequence of actions with one input leading to multiple outputs over time (see Fig. 2).

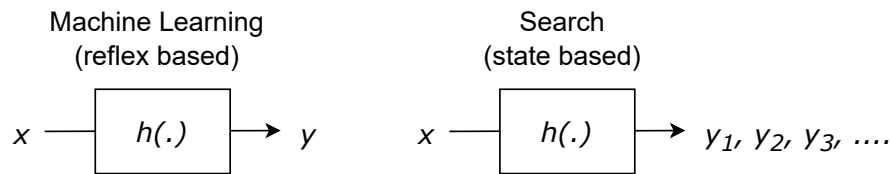


Figure 2: Machine Learning: single action. Search: sequence of actions.

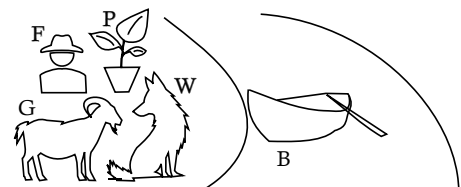
For example, consider a pathfinding problem. Suppose you're in Gulshan and want to reach Dhanmondi. Your goal might be to find the shortest path (objective). However, priorities can change depending on a situation – maybe you now care more about finding the fastest path (another objective), even if it's not the shortest. The solution depends on the defined objective, and we need to consider the future impact of each action.

For instance, an empty road might seem like a good choice initially (locally optimal). But if it leads to a road with heavy traffic due to construction, it's a poor decision overall (globally suboptimal). The key is to evaluate actions not just in the moment but also based on their future consequences. How can we do that?

A river crossing problem

Consider this problem. A farmer (F) needs to cross a river with three of his companions: a Goat (G), a Plant (P), and a Wolf (W). He can go back and forth as many times as he wants, but for each crossing, he has to pay \$1. He is also a little low in cash, which is why he wants to get to the other side of the river with the minimum expense possible.

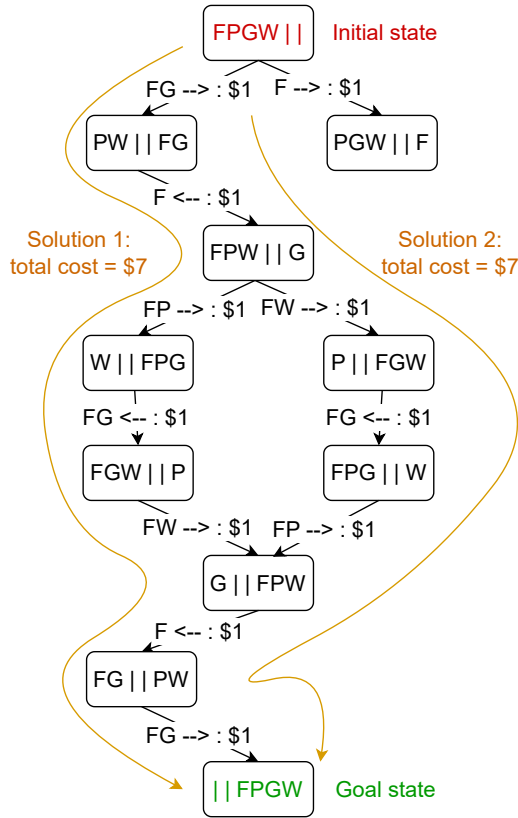
The farmer's small boat can only carry him (F) and one item – either the G , the P , or the W – at a time. The situation is tricky: If the G is left alone with the P , it will eat the P . If the W is left alone with the G , it will attack the G . What should he do?



Search tree: starts from the initial state, then branches out based on all possible actions. A search tree is a structure where each node represents a state, and the branches represent possible moves or actions that lead to other states. Each action has a cost (\$1 in this case). Any path that takes from the initial state to the goal state is a valid path. The path that corresponds to the minimum total cost is the solution. Can you now think of an algorithm that can provide these solutions?

A transportation problem

Imagine you are walking along a street divided into blocks, starting at block 1 and aiming to reach block N . At each block, you have two options to move forward: a) you can either walk to the next block, which costs 1 unit, or b) bike to the next block, which costs 2 units. Your goal is to reach block



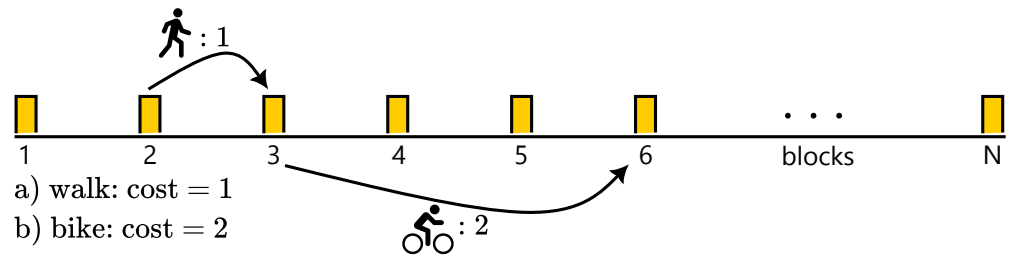
In this problem, we start with everyone on the left side of the river (initial state: $FPGW||$), which is the initial state. The goal is to get everyone safely to the right side of the river (goal state: $||FPGW$). To achieve this, we need to consider all the possible valid actions. For example, the farmer can go alone from left to right ($F \rightarrow$), or he can take the goat with him from left to right ($FG \rightarrow$). Similarly, the farmer can take the plant back with him from right to left ($FP \leftarrow$), and so on. The complete list of valid actions is: $\mathcal{A} = \{F \rightarrow, FP \rightarrow, FG \rightarrow, FW \rightarrow, F \leftarrow, FP \leftarrow, FG \leftarrow, FW \leftarrow\}$.

Note that $FPG \rightarrow$ is not, for example, a valid action, as it breaks the condition stated in the problem. Also, actions in $\mathcal{A}$ are not valid from every state and some may lead to an invalid state, meaning they could result in a state that violates the conditions.

As you can see in the tree above, there are two paths from the initial state to the goal state. Both paths have the same total cost (\$7), so either path can be considered the solution.

N while minimizing the total cost. To represent this, think of each block as a state s . At each state, you must choose whether to walk or bike, determining the sequence of actions that leads to the lowest total cost by the time you arrive at your destination, block N .

Walking takes from $s \rightarrow s + 1$, biking takes from $s \rightarrow 2s$.
Objective: Go from $1 \rightarrow N$ with minimum total cost.



Say, $N = 10$. One possible solution: $walk \rightarrow walk \rightarrow walk \rightarrow walk \rightarrow bike$, $\sum cost = 6$. Another possible solution: $walk \rightarrow bike \rightarrow walk \rightarrow bike$, $\sum cost = 6$. Not a solution: $bike \rightarrow bike \rightarrow bike \rightarrow walk \rightarrow walk$, as $\sum cost = 8 > 6$ (minimum cost).

Algorithms for search

Backtracking search

Backtracking search is one of the simplest and probably one of the most intuitive algorithms that comes to mind when solving a search tree. It's a straightforward method used to explore all possible solutions in a search tree. The goal of backtracking is to systematically explore this tree to find a solution, if one exists.

The process starts at the root (or initial state) of the search tree and explores one branch at a time. If you reach a point where no solution is possible (like a dead end), you “backtrack” to the previous node and try a different branch. This way, you explore the tree depth-first, checking all possible paths one by one. It's like trying all the keys on a keyring one at a time.

Breadth First Search (BFS)

Breadth-First Search (BFS) is a tree search algorithm that explores all nodes at the same depth level before moving to the next level. It systematically examines the search tree level by level, starting from the root and exploring all neighboring nodes before progressing deeper. This makes BFS a great choice for finding the shortest path in an unweighted graph or when searching for a solution that is closest to the root. A flowchart for the BFS algorithm is provided in Fig. 3.

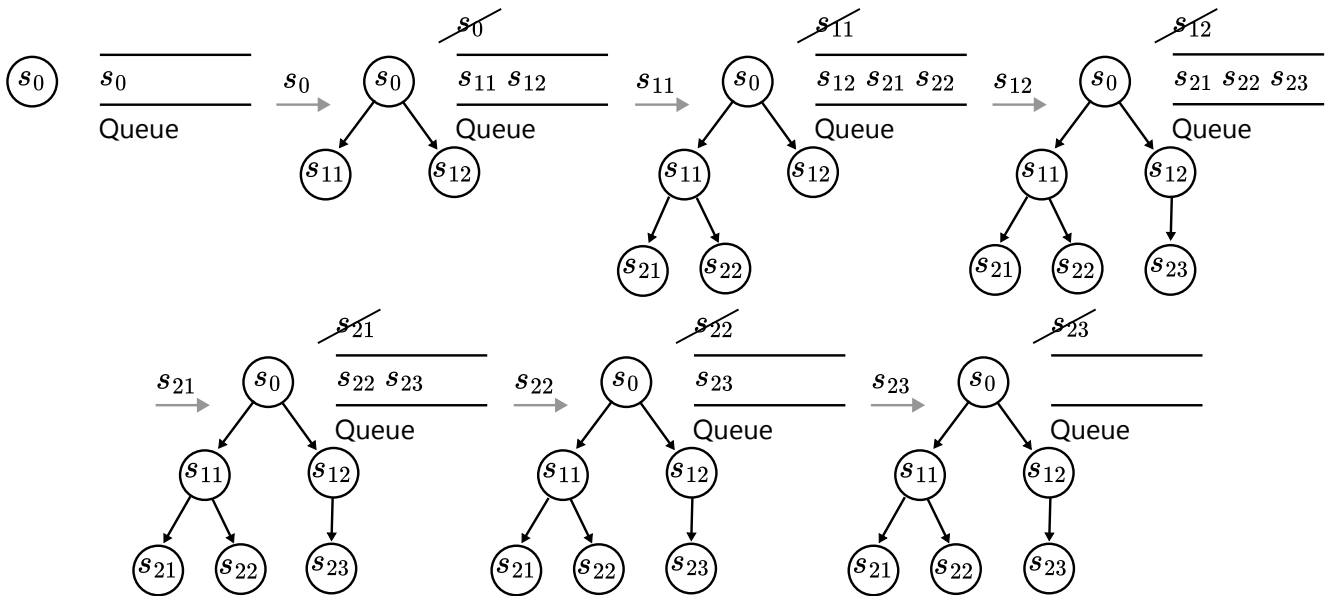


Figure 3: BFS Explore Sequence: $s_0 \rightarrow s_{11} \rightarrow s_{12} \rightarrow s_{21} \rightarrow s_{22} \rightarrow s_{23}$.

The algorithm uses a **queue** data structure to keep track of nodes that need to be explored. It begins by adding the starting node to the queue. Then, in each step, it removes the front node from the queue, examines it, and adds its unvisited neighbors to the back of the queue. This ensures that nodes are visited in order of their depth in the tree. While BFS guarantees finding the shortest path in terms of the number of steps, it can consume significant memory, especially for trees or graphs with many nodes at each level.

Depth First Search (DFS)

Depth-First Search (DFS) is an algorithm that explores a tree or graph by going as deep as possible along each branch before backtracking. It starts at the root and explores one path completely, moving down to the next node and continuing until it reaches a dead end or a solution. When it hits a dead end, DFS backtracks to the last decision point and tries a different path. A flowchart for the DFS algorithm is provided in Fig. 4.

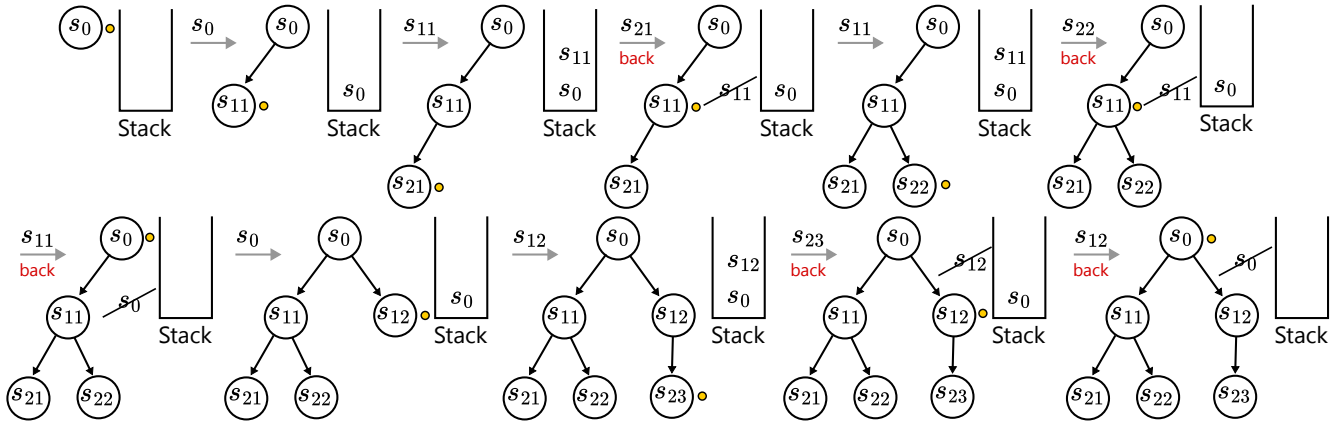


Figure 4: DFS Explore Seq.: $s_0 \rightarrow s_{11} \rightarrow s_{21} \rightarrow s_{11} \rightarrow s_{22} \rightarrow s_{11} \rightarrow s_0 \rightarrow s_{12} \rightarrow s_{23} \rightarrow s_{12} \rightarrow s_0$.

DFS uses a [stack](#) to keep track of which nodes to visit next. It starts by adding the starting node to the stack, then picks the top node, checks it, and adds its neighbors to the stack. The algorithm continues down the current path until it can't go further, then it pops the stack to go back and explore other branches. DFS is useful when you need to explore all possible solutions or paths, but it doesn't always find the shortest path and can take up a lot of memory if the tree is very deep.

Depth-Limited Search and Iterative Deepening DFS

Depth-Limited Search (DLS) is a variation of Depth-First Search (DFS) where the algorithm is restricted to exploring nodes only up to a certain depth, preventing it from going too deep and potentially using excessive memory or time. This helps avoid getting stuck in infinite loops or exploring overly deep paths in large search spaces. However, the limitation is that DLS might miss the solution if it's deeper than the specified limit.

Iterative Deepening Depth-First Search (IDDFS) combines the advantages of DFS and DLS by performing multiple depth-limited searches with increasing depth limits. It starts by performing a DFS with a depth limit of 1, then repeats the search with a limit of 2, then 3, and so on, until it finds the solution. This approach ensures that IDDFS will eventually explore all nodes, similar to BFS, but without requiring the large memory overhead. IDDFS is particularly useful when the solution is unknown, and memory is limited, as it combines the memory efficiency of DFS with the completeness of BFS.

Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is an algorithm used to find the shortest path in a weighted graph, where the cost of moving between nodes may differ. Unlike BFS, which assumes all steps have the same cost, UCS takes the path cost into account and always explores the least costly path first. It does this by

expanding the node with the lowest cumulative cost, ensuring that the first time it reaches a node, it does so with the least possible cost. A flowchart for the UCS algorithm is provided in Fig. 5.

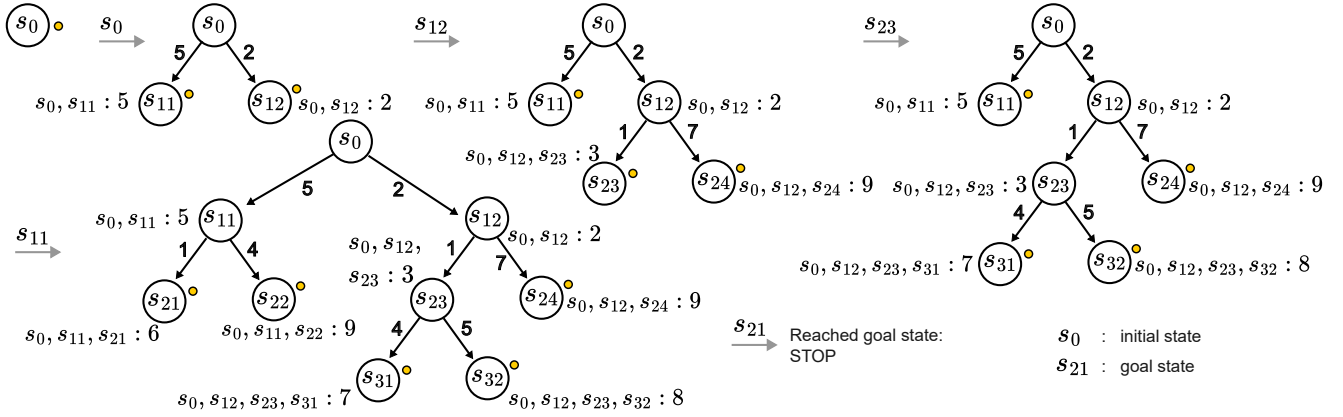


Figure 5: UCS Explore Sequence: $s_0 \rightarrow s_{12} \rightarrow s_{23} \rightarrow s_{11} \rightarrow s_{21}$. Solution: $s_0, s_{11}, s_{21} : 6$.

UCS uses a priority queue (often implemented as a min-heap) to keep track of the nodes to be explored. Each node in the queue is prioritized based on its path cost, and the algorithm removes the node with the lowest cost to explore it further. UCS guarantees finding the optimal solution if the graph has non-negative edge weights, but it can be slower and require more memory than algorithms like BFS, especially in graphs with many nodes or high branching factors. It is ideal for problems where the goal is to find the least-cost path, such as in route planning or network optimization.

A* Search

A* Search is an algorithm that combines the strengths of Uniform Cost Search (UCS) with the concept of heuristics to find the most efficient path in a graph. Like UCS, A* ensures that it explores the least costly path by considering the cumulative cost of reaching a node. However, A* also uses a heuristic, which is an estimate of the remaining cost to reach the goal from a node. This helps guide the search more efficiently toward the goal. In A*, path costs are modified (penalized) based on heuristic values of each node:

$$C(s_1 \rightarrow s_2) = C_0(s_1 \rightarrow s_2) + h(s_2) - h(s_1), \quad h(s) = \text{heuristic value at state } s.$$

Here, $C(\cdot)$ denotes the new cost and $C_0(\cdot)$ denotes the old cost. Consider the tree on the left. Let's say the heuristic values are given. Using the modified cost calculations, we redraw the tree (see the tree on the right).

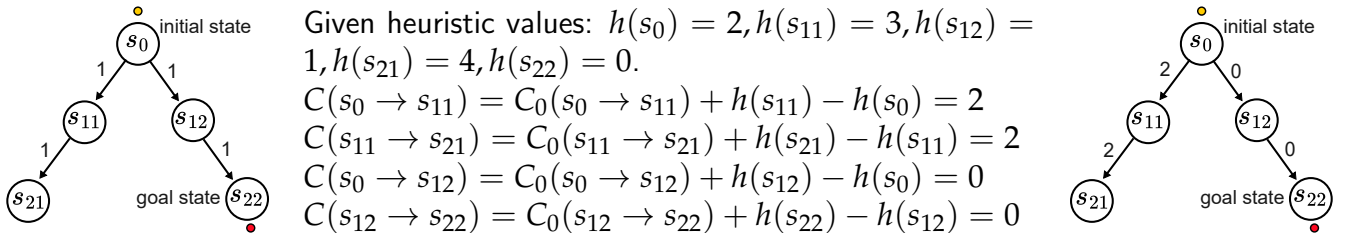


Figure 6: UCS on the modified tree with new costs will now be more efficient.

A* uses a priority queue, where each node is assigned a priority based on the sum of two factors: the path cost to reach the node (like UCS) and the heuristic estimate of the cost to reach the goal. The algorithm always expands the node with the lowest total cost (path cost + heuristic). This combination of cost and heuristic allows A* to prioritize paths that are both promising and efficient, making it faster than UCS in many cases. A* is guaranteed to find the optimal solution if the heuristic is admissible (it never overestimates the true cost). It is widely used in route planning, game AI, and other optimization problems.

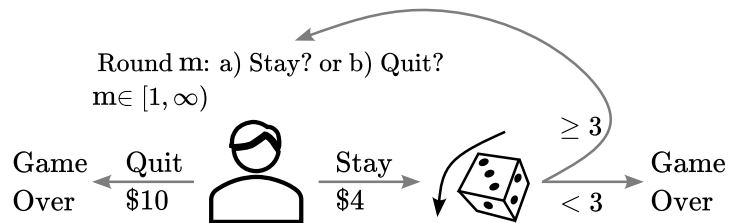
Markov decision process (MDP)

A dice game: stay or quit?

How can you get to Mountain View on Friday night in the shortest time? You could bike, drive, take a train, call an Uber, or fly. Each option, however, comes with uncertainties. For instance, if you decide to drive, you might encounter traffic.

Traditional search problems are typically deterministic, assuming no uncertainty in outcomes. Real-world scenarios like this one require a probabilistic or stochastic approach to account for the inherent uncertainties. How can we adapt this problem to include these considerations? Now, say you are playing a game, here are the rules:

For each round $m = 1, 2, \dots$, you can stay or quit. If quit, you will get \$10; if stay, you will get \$4 and I roll a 6-sided dice. If the dice results in 1 or 2, the game is over. Otherwise, you can move to the next round. What would you do?



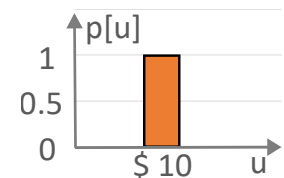
Optimum policy (π)

Imagine you're starting the game in Round 1. At this point, what's the best policy (π)? Should you stay or quit? To answer this, we first need to define what we mean by 'best.' A policy is considered the best if it maximizes the rewards you accumulate in the game. Let's introduce some terminology to formalize this idea. Each choice you make results in a reward, denoted as r , and the total reward you collect over all rounds is called the utility, u , expressed as $u = \sum r$. The best policy, then, is the one that maximizes u .

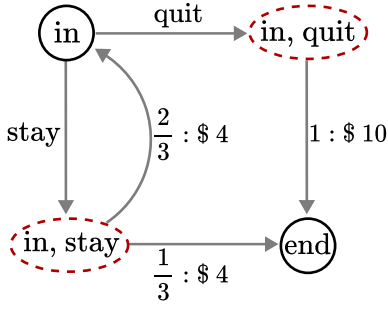
However, the game is stochastic, meaning there's an element of randomness. Even if you follow the same policy π , the resulting u may differ because it's a random variable (therefore, let's denote utility as U). To account for this uncertainty, we consider the expected utility, denoted as $E[U]$, and determine which policy maximizes this expectation.

To clarify further, let's introduce a parameter called the value, denoted as v , where $v = E[U]$. Thus, the task of finding the best policy boils down to maximizing v with respect to the policy π . The policy that achieves this is what we call the optimal policy for the current state. For the choice quit, we can easily find v , but for the choice stay, not that easy.

Let's say our policy is to **quit**. In this case, we can define a probability mass function (PMF) for U , as the probability of $U = 10$ is always 1. Therefore, $v = E[U] = \sum uP(U = u) = \sum up[u] = 1 \times \$10 = \$10$. Okay, what if our policy now is to **stay**. Do we have a PMF for it? Actually we don't (at least not easy to find). Then how will we calculate v now?



Let us denote the dice outcome as a random variable D with domain (sample space) $\mathcal{D} = \{1, 2, 3, 4, 5, 6\}$. Let us pay attention to the following figure. It represents the state-diagram of the problem. The probability of obtaining 1 or 2, i.e., $P(D < 3) = 2/6 = 1/3$. Similarly, the probability of obtaining 3 or higher, i.e., $P(D \geq 3) = 4/6 = 2/3$.



Let us denote the transition probability and reward by $T(s, a, s')$ and $R(s, a, s')$, where s is the current state (either 'in' or 'end'), s' is the next state (either 'in' or 'end'), a is the action chosen (either 'stay' or 'quit') that takes you from s to s' . Therefore,
 $T(in, stay, end) = 1/3$, $T(in, quit, end) = 1$, $T(in, stay, in) = 2/3$ and $R(in, stay, end) = \$ 4$, $R(in, quit, end) = \$ 10$, $R(in, stay, in) = \$ 4$.

Policy evaluation

The value of a policy π at state s can be calculated by

$$v_{\pi}(s) = \begin{cases} 0, & \text{end state} \\ Q_{\pi}(s, a), & \text{otherwise} \end{cases}; \quad Q_{\pi}(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma v_{\pi}(s')]$$

where s is the current state, s' is the next state, a is the action chosen that takes from s to s' . $T(\cdot)$ and $R(\cdot)$ are the transition probabilities and rewards (mentioned before). γ here is called the discount factor. For now, we will hold $\gamma = 1$. Now let's evaluate the policies for our dice game above using this formula. When the policy is to stay ($\pi = a = \text{stay}$), $v_{\pi}(in) = Q_{\pi}(in, \text{stay})$. Therefore,

$$\begin{aligned} v_{\pi=\text{stay}}(in) &= \sum_{s'} T(in, \text{stay}, s') [R(in, \text{stay}, s') + v_{\pi=\text{stay}}(s')] \\ &= T(in, \text{stay}, in) [R(in, \text{stay}, in) + v_{\pi=\text{stay}}(in)] + T(in, \text{stay}, end) [R(in, \text{stay}, end) + v_{\pi=\text{stay}}(end)] \\ &= \frac{2}{3} [\$ 4 + v_{\pi=\text{stay}}(in)] + \frac{1}{3} [\$ 4 + 0] = \$ 4 + \frac{2}{3} v_{\pi=\text{stay}}(in) \end{aligned}$$

Therefore, $v_{\pi=\text{stay}}(in) = \$ 12$. This means the value v for policy *stay* at state 'in' is \$ 12. Let's do the same for policy quit ($\pi = a = \text{quit}$). We calculated this a little before using PMF and the value v came out to be \$ 10. Let's try to find it again using this new formula and see if it matches the old result:

$$\begin{aligned} v_{\pi=\text{quit}}(in) &= \sum_{s'} T(in, \text{quit}, s') [R(in, \text{quit}, s') + v_{\pi=\text{quit}}(s')] \\ &= T(in, \text{quit}, in) [R(in, \text{quit}, in) + v_{\pi=\text{quit}}(in)] + T(in, \text{quit}, end) [R(in, \text{quit}, end) + v_{\pi=\text{quit}}(end)] \\ &= 0 [\$ 0 + v_{\pi=\text{quit}}(in)] + 1 [\$ 10 + 0] = \$ 10 \end{aligned}$$

Therefore, $v_{\pi=\text{quit}}(in) = \$ 10$. This means the value v for policy *quit* at state 'in' is \$ 10, which matches the old result calculated using PMF.

Optimum policy

The optimum policy $\pi^*(in) = \arg \max_{\pi} v_{\pi}(in)$. As $v_{\pi=\text{stay}}(in) > v_{\pi=\text{quit}}(in)$ (\$ 12 > \$ 10), the optimum policy $\pi^*(in) = \text{stay}$ at state 'in'.

Value iteration

Iterative solution. For policy evaluation, begin by initializing $v_{\pi=a}(s)$ randomly (or maybe with just zeros). Then perform the following update:

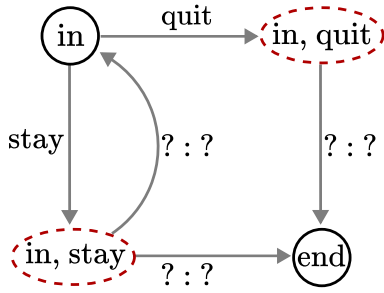
$$v_{\pi=a}(s) := Q_{\pi=a}(s, a), \text{ stopping criteria: } \Delta v_{\pi=a}(s) < \epsilon.$$

For calculating the optimum value v^* and optimum policy π^* , perform the following update:

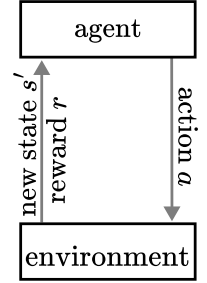
$$v^*(s) := \max_a Q_{\pi=a}(s, a), \quad \pi^*(s) := \arg \max_a Q_{\pi=a}(s, a), \quad \text{stopping criteria: } \Delta v^*(s) < \epsilon.$$

Reinforcement learning (RL)

Consider the dice game again. The transition probabilities and reward values, denoted as T and R , were given earlier in the problem statement. But what if these values are not provided, and we need to find them ourselves? In that case, they can be treated as model parameters within the MDP framework. Now the model parameters are calculated during the learning (training) phase. This implies that we need training data to estimate these parameters. But where does this training data come from?



Imagine someone already played the game, and you observed their actions: the choices they made, the outcomes of the dice rolls, and the rewards they received. These observations form your training data (called episode) is represented as $s_0; a_1 r_1 s_1; a_2 r_2 s_2; \dots; end$, where s_{i-1} is the current state and s_i is the next state reached due to action a_i obtaining reward r_i .



Mote Carlo method (model-based)

Estimate transition probabilities and reward

$$\hat{T}(s, a, s') = \frac{\# \text{ times } (s, a, s') \text{ occurs}}{\# \text{ times } (s, a) \text{ occurs}}; \quad \hat{R}(s, a, s') = r \text{ in } (s, a, r, s')$$

Imagine, we are given the episode data: $e_1 = in; stay, \$4, in; stay, \$4, in; stay, \$4, end$.

Based on this data, $\hat{T}(in, stay, in) = 3/4$, $\hat{R}(in, stay, in) = \4 , $\hat{T}(in, stay, end) = 1/4$, $\hat{R}(in, stay, end) = \4 , $\hat{T}(in, quit, end) = \text{cannot find}$, $\hat{R}(in, quit, end) = \text{cannot find}$ (absent in episode).

Now imagine you got two more episode data: $e_2 = in; stay, \$4, in; stay, \$4, end$, $e_3 = in; stay, \$4, end$. You need to update the estimates considering all of e_1, e_2, e_3 . The new estimates become $\hat{T}(in, stay, in) = 4/7$, $\hat{R}(in, stay, in) = \4 , $\hat{T}(in, stay, end) = 3/7$, $\hat{R}(in, stay, end) = \4 .

Model free Mote Carlo

Estimate $Q_\pi \rightarrow \hat{Q}_\pi$.

$$\hat{Q}_{opt}(s, a) = \sum_{s'} \hat{T}(s, a, s') [\hat{R}(s, a, s') + \gamma \hat{v}_{opt}(s')]$$

$$u = \sum_i r_i \implies \text{utility}$$

$$\text{training data} \rightarrow in; stay, 4, end. \quad u = 4$$

$$\text{training data} \rightarrow in; stay, 4, in; stay, 4, end. \quad u = 4 + 4 = 8$$

$$\text{training data} \rightarrow in; stay, 4, \dots, \dots, end. \quad u = 4 + 4 + 4 + 4 = 16$$

$$\text{average } u = \frac{4 + 8 + 16}{3} = \hat{Q}_\pi$$

Botstrapping

$$\hat{Q}_{\pi}(s, a) \leftarrow (1 - \eta)\hat{Q}_{\pi}(s, a) + \eta[r + \gamma\hat{Q}_{\pi}(s', a')] \quad \text{SARSA}$$

Also estimates $\hat{Q}_{\pi}$ not $\hat{Q}_{opt}$.

Q-learning

Allows us to get $\hat{Q}_{opt}$ (not $\hat{Q}_{\pi}$).

On each (s, a, r, s')

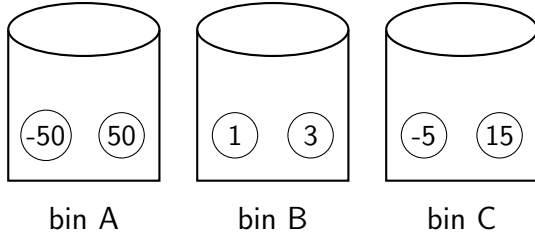
$$\begin{aligned}\hat{Q}_{opt} &\leftarrow (1 - \eta)\hat{Q}_{opt}(s, a) + \eta[r + \gamma\hat{V}_{opt}(s')] \\ \hat{v}_{opt}(s') &= \max_{a' \in \text{Action set}} \hat{Q}_{opt}(s', a')\end{aligned}$$

Deep reinforcement learning - use neural net to estimate $\hat{Q}_{opt}$.

This volume: Game + Factor graphs

A simple turn-taking game

Consider this simple turn-based game. See how you decide your actions.



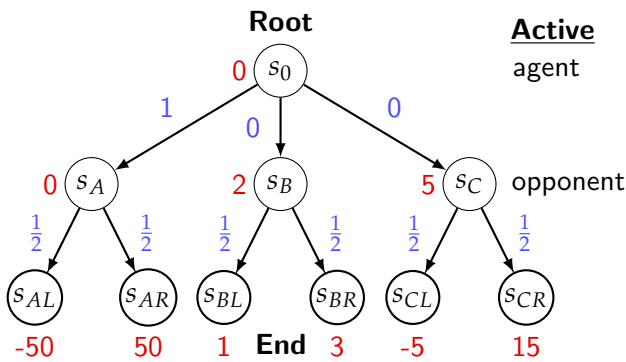
There are 3 bins – A, B, & C. Each bin has 2 balls (left & right). The numbers on the balls represent reward points. You're the agent, and I'm your opponent. First, you choose a bin. Then, I pick one of the 2 balls from that bin, and you'll get the reward points written on it. So, which bin would you choose?

Game evaluation

Both π_{ag} and π_{opp} given. Value of the game is given as

$$V(s) = \begin{cases} utility(s), & \text{end state} \\ \sum_a \pi_{ag}(s, a) V(s'), & \text{agent active} \\ \sum_a \pi_{opp}(s, a) V(s'), & \text{opponent active} \end{cases}$$

Say for example, $\pi_{ag}(s) = [1, 0, 0]$ and $\pi_{opp}(s) = [1/2, 1/2]$. In other words, $\pi_{ag}(s, A) = 1$, $\pi_{ag}(s, B) = 0$, $\pi_{ag}(s, C) = 0$, $\pi_{opp}(s, L) = 1/2$ and $\pi_{opp}(s, R) = 1/2$.



Here, $\pi_{ag}(s) = [1, 0, 0]$, $\pi_{opp}(s) = [1/2, 1/2]$

$$V(s_{AL}) = utility(s_{AL}) = -50$$

$$V(s_B) = \sum_a \pi_{opp}(s_B, a) V(s')$$

$$= \pi_{opp}(s_B, L) V(s_{BL}) + \pi_{opp}(s_B, R) V(s_{BR}) = 2$$

$$V(s_0) = \sum_a \pi_{ag}(s_0, a) V(s') = 0$$

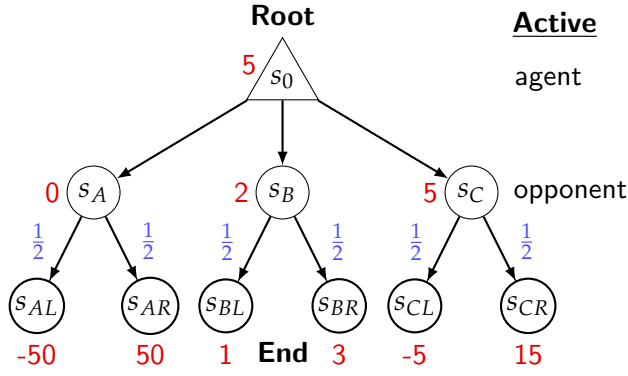
$$= \pi_{ag}(s_0, A) V(s_A) + \pi_{ag}(s_0, B) V(s_B) + \pi_{ag}(s_0, C) V(s_C)$$

Expectimax policy

π_{ag} not given (need to find out) and π_{opp} is assumed. Value of the game is given as

$$V(s) = \begin{cases} utility(s), & \text{end state} \\ \max_a V(s'), & \text{agent active} \\ \sum_a \pi_{opp}(s, a) V(s'), & \text{opponent active} \end{cases} \quad \pi_{ag}^*(s) = \arg \max_a V(s')$$

Say our assumed $\pi_{opp}(s) = [1/2, 1/2]$.



Here, $\pi_{ag}(s) = [1, 0, 0]$, $\pi_{opp}(s) = [1/2, 1/2]$

$$V(s_{BL}) = utility(s_{BL}) = 1$$

$$V(s_C) = \sum_a \pi_{opp}(s_C, a) V(s')$$

$$= \pi_{opp}(s_C, L) V(s_{CL}) + \pi_{opp}(s_C, R) V(s_{CR}) = 5$$

$$V(s_0) = \max_a V(s')$$

$$= \max(V(s_A), V(s_B), V(s_C)) = 5$$

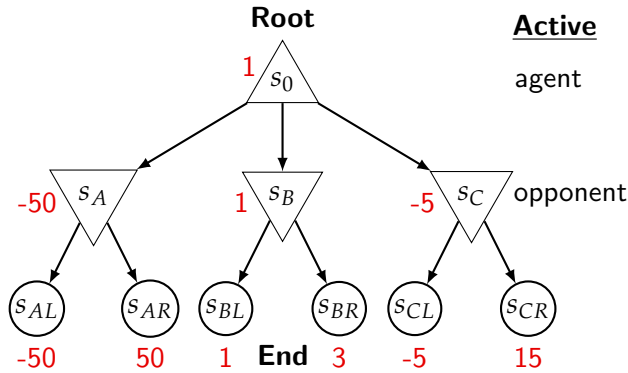
Minimax policy

π_{ag} not given (need to find out) and π_{opp} not given (need to find out). But motives given for both agent and opponent. Value of the game is then given as

$$V(s) = \begin{cases} utility(s), & \text{end state} \\ \max_a V(s'), & \text{agent active} \\ \min_a V(s'), & \text{opponent active} \end{cases}$$

$$\pi_{ag}^*(s) = \arg \max_a V(s'), \pi_{opp}^*(s) = \arg \min_a V(s')$$

For the same game above,



Here, $\pi_{ag}(s) = [1, 0, 0]$, $\pi_{opp}(s) = [1/2, 1/2]$

$$V(s_{BL}) = utility(s_{BL}) = 1$$

$$V(s_C) = \sum_a \pi_{opp}(s_C, a) V(s')$$

$$= \pi_{opp}(s_C, L) V(s_{CL}) + \pi_{opp}(s_C, R) V(s_{CR}) = 5$$

$$V(s_0) = \max_a V(s')$$

$$= \max(V(s_A), V(s_B), V(s_C)) = 5$$

CSP algorithm

input $\rightarrow$ partial assignment x , current domain of unassigned variables $\mathcal{D}$

if x is complete

return;

else

Choose an assigned variable X_i

For each value v in $\mathcal{D}_i$ of chosen X_i

$$\delta := \prod f_j(x \cup \{X_i : v\})$$

if $\delta = 0$

continue;

else

update $\mathcal{D}_i$ of unassigned vars of $X_i : v$
repeat with partial assignment $x \cup \{X_i : v\}$ & update domain

Volume 4

This volume: Bayesian + Logic

Bayesian Network

References